Experiment 3 : SVM Classification of Handwritten Dig

Aim: Classify images of handwritten digits (0 - 9) using a Support Vector Machine.

In [1]:
```
# A52 Dipanshu Ambilkar
from sklearn.datasets import load_digits
from sklearn.model_selection import train_test_split
from sklearn.svm import SVC
from sklearn.metrics import accuracy_score, classification_report, confus
```

In [2]:
```
X, y = load_digits(return_X_y=True)
print("Shape:", X.shape, "| classes:", sorted(set(y)))
```

```
Shape: (1797, 64) | classes: [np.int64(0), np.int64(1), np.int64(2), np.int64
np.int64(4), np.int64(5), np.int64(6), np.int64(7), np.int64(8), np.int64(9)]
```

In [3]:
```
print("First 8 pixel values of the first image:", X[0][:8])
print("Label of the first image:", y[0])
```

```
First 8 pixel values of the first image: [ 0.  0.  5. 13.  9.  1.  0.  0.]
Label of the first image: 0
```

In [4]:
```
X_train, X_test, y_train, y_test = train_test_split(
    X, y, test_size=0.2, random_state=7)
print("Train size:", len(X_train), "| Test size:", len(X_test))
```

```
Train size: 1437 | Test size: 360
```

In [5]:
```
model = SVC()
print("Model created:", model)
```

```
Model created: SVC()
```

In [6]:
```
model.fit(X_train, y_train)
print("SVM trained")
```

```
SVM trained
```

In [7]:
```
pred = model.predict(X_test)
print("First 10 predictions:", pred[:10])
print("First 10 actual labels:", y_test[:10])
```

```
First 10 predictions: [6 0 5 9 2 9 0 4 1 0]
First 10 actual labels: [6 0 5 9 2 9 0 4 1 0]
```

In [8]:
```
print("Accuracy:", round(accuracy_score(y_test, pred), 4))
```

```
Accuracy: 0.9889
```

In [9]:
```
print("Classification report:")
print(classification_report(y_test, pred))
```

```
Classification report:
```

	precision	recall	f1-score	support
0	1.00	1.00	1.00	43
1	0.95	1.00	0.98	42
2	1.00	1.00	1.00	40
3	1.00	1.00	1.00	34
4	1.00	1.00	1.00	37
5	0.93	1.00	0.97	28
6	1.00	1.00	1.00	28
7	1.00	1.00	1.00	33
8	1.00	0.93	0.96	43
9	1.00	0.97	0.98	32
accuracy			0.99	360
macro avg	0.99	0.99	0.99	360
weighted avg	0.99	0.99	0.99	360

In [10]:
```
print("Confusion matrix:")
print(confusion_matrix(y_test, pred))
```

```
Confusion matrix:
[[43  0  0  0  0  0  0  0  0  0]
 [ 0 42  0  0  0  0  0  0  0  0]
 [ 0  0 40  0  0  0  0  0  0  0]
 [ 0  0  0 34  0  0  0  0  0  0]
 [ 0  0  0  0 37  0  0  0  0  0]
 [ 0  0  0  0  0 28  0  0  0  0]
 [ 0  0  0  0  0  0 28  0  0  0]
 [ 0  0  0  0  0  0  0 33  0  0]
 [ 0  2  0  0  0  1  0  0 40  0]
 [ 0  0  0  0  0  1  0  0  0 31]]
```